Bit-Mapping bei Schieberegistern

CD4021B (Input) & 74HC595 (Output)

„MSB vs. LSB – Warum die Asymmetrie?“

ESP32-S3 ● Selection Panel ● SPI-Kommunikation

Keywords

CD4021B • 74HC595 • PISO/SIPO • MSB/LSB • Bit-Masken

Stand: 20.01.2026
Version 1.0.0

Jan Unger
Tools: Claude Opus 4.5, ChatGPT 5.2

Inhaltsverzeichnis

1 Zahlensysteme und Bit-Operationen – Grundlagen **1**
1.1 Begriffsrahmen und Zielsetzung . . . 1
1.2 Zahlensysteme – Drei Darstellungen, ein Wert . . . 1
1.3 Bit-Positionen – Wo ist was im Byte? . . . 2
1.4 Datentypen – Die richtige Größe wählen . . . 3
1.5 Bit-Operationen – Die vier Grundoperationen . . . 4
1.6 Warum `1u` statt `1`? . . . 6
1.7 Maske erstellen – Vom Bit zur Maske . . . 6
1.8 MSB-first Mapping – Von der ID zur Bit-Position . . . 6
1.9 Praktische Funktionen . . . 7
1.10 Last-Byte-Maskierung . . . 8
1.11 Zusammenfassung – Merkregeln . . . 9
1.12 Weiterführende Dokumentation . . . 10

2 Bit-Mapping – Schieberegister verstehen **10**
2.1 Begriffsrahmen und Zielsetzung . . . 10
2.2 Das Grundproblem – IDs vs. Bytes . . . 11
2.3 MSB-first – Warum Bit 7 zuerst? . . . 11
2.4 Buttons (CD4021) – Active-Low lesen . . . 12
2.5 LEDs (74HC595) – Active-High schreiben . . . 13
2.6 Last-Byte-Maskierung – Ungültige Bits behandeln . . . 14
2.7 Kaskadierung – Mehrere ICs verketten . . . 15
2.8 Vollständiges Beispiel – Button 7 drücken . . . 16
2.9 Skalierung – 10 → 100 ohne Code-Änderung . . . 17
2.10 Fehlerbehebung . . . 17
2.11 Weiterführende Dokumentation . . . 18

1 Zahlensysteme und Bit-Operationen – Grundlagen

1.1 Begriffsrahmen und Zielsetzung

Bei der Arbeit mit Schieberegistern (CD4021, 74HC595) manipulierst du einzelne Bits in Bytes. Dafür brauchst du drei Fähigkeiten: Zahlensysteme verstehen, Datentypen korrekt wählen, Bit-Operationen sicher anwenden.

Begriff	Definition
Bit	Kleinste Informationseinheit (0 oder 1)
Byte	8 Bits, Wertebereich 0–255
Nibble	4 Bits, entspricht einer Hex-Ziffer
MSB	Most Significant Bit – Bit 7, höchstwertiges Bit
LSB	Least Significant Bit – Bit 0, niedrigstwertiges Bit
Maske	Bit-Muster zur gezielten Manipulation bestimmter Bits

Tabelle 1: Begriffsrahmen Bit-Operationen

Ziel: Du kannst eine Button-ID (z. B. 7) in eine Bit-Position umrechnen und das entsprechende Bit lesen oder setzen.

1.2 Zahlensysteme – Drei Darstellungen, ein Wert

Problem: Der Wert „zwölf“ lässt sich auf verschiedene Arten schreiben. In der Embedded-Entwicklung begegnen dir alle drei Darstellungen – oft im selben Code.

System	Basis	Ziffern	C/C++ Präfix	Beispiel (12)
Dezimal	10	0–9	(keiner)	`12`
Binär	2	0, 1	`0b`	`0b00001100`
Hexadezimal	16	0–9, A–F	`0x`	`0x0C`

Tabelle 2: Zahlensysteme im Vergleich

Wann verwendest du welches System?

- **Dezimal** für Zählwerte, Indizes, Zeitangaben: `count = 10`
- **Binär** wenn du einzelne Bits siehst: `0b10000000` (Bit 7 gesetzt)
- **Hex** für Byte-Werte und Adressen: `0x80` (kompakter als binär)

Nibble-Prinzip – Hex lesen lernen

Ein Byte besteht aus zwei Nibbles (je 4 Bits). Jedes Nibble entspricht einer Hex-Ziffer:

```
Byte:     0b1100.0101
```

```
Nibbles:    C     5
Hex:      0xC5
```

Umrechnungstabelle (auswendig lernen lohnt sich):

Hex	Binär	Dez	Hex	Binär	Dez
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	A	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

Tabelle 3: Hex-Binär-Dezimal Umrechnungstabelle

Beispiel – 0x80 in Binär umwandeln:

```
0x80 = 0x8 + 0x0
     = 0b1000 + 0b0000
     = 0b10000000
```

Das ist Bit 7 gesetzt – genau die Maske für ID 1 im MSB-first-Mapping.

1.3 Bit-Positionen – Wo ist was im Byte?

Aufbau eines Bytes:

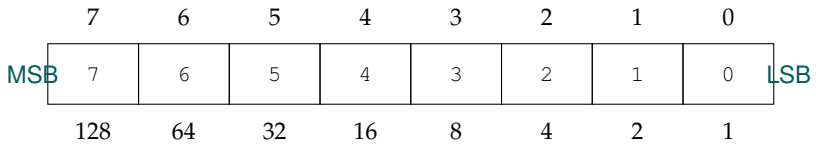


Abbildung 1: Aufbau eines Bytes mit Bit-Positionen und Stellenwerten

Stellenwert-Formel: Bit n hat den Wert 2^n.

Beispiel – Welchen Wert hat `0b01001010`**?**

```
Gesetzte Bits: 6, 3, 1
Wert = 64 + 8 + 2 = 74 = 0x4A
```

Bit	Stellenwert	Hex-Maske
7	128	`0x80`
6	64	`0x40`
5	32	`0x20`
4	16	`0x10`
3	8	`0x08`
2	4	`0x04`
1	2	`0x02`
0	1	`0x01`

Tabelle 4: Stellenwerte und Hex-Masken

1.4 Datentypen – Die richtige Größe wählen

Problem: C/C++ bietet viele Integer-Typen. Bei Embedded-Systemen ist die exakte Bitbreite entscheidend – ein Schieberegister liefert genau 8 Bits, nicht mehr, nicht weniger.

Typen mit fester Breite (aus `<stdint.h>`):

Typ	Bits	Wertebereich	Typischer Einsatz
`uint8_t`	8	0 – 255	Schieberegister-Bytes
`int8_t`	8	−128 – +127	Vorzeichen-behaftete kleine Werte
`uint16_t`	16	0 – 65.535	Sequenznummern, Timer
`uint32_t`	32	0 – 4.294.967.295	Timestamps, große Zähler

Tabelle 5: Integer-Typen mit fester Breite

Namenskonvention: `u` = unsigned, Zahl = Bits, `_t` = type

Warum `uint8_t` für Schieberegister?

```
// Korrekt: exakt 8 Bit
uint8_t buttonState = 0;
uint8_t ledState[2] = {0, 0};

// Problematisch: Groesse plattformabhaengig
int buttonState = 0;  // 16 oder 32 Bit - verschwendet RAM
```

Überlauf beachten

```
uint8_t x = 255;
x = x + 1;  // x = 0, nicht 256!

uint8_t y = 0;
y = y - 1;  // y = 255, nicht -1!
```

Typ	Empfehlung
`uint8_t`	Empfohlen – exakt 8 Bit, portabel
`byte`	Nur Arduino – nicht portabel
`unsigned char`	Funktioniert, aber weniger explizit
`int`	Zu groß, verschwendet RAM

Tabelle 6: Typ-Empfehlungen für Byte-Daten

Regel

Bei `uint8_t` „wickelt“ der Wert um – von 255 auf 0 bzw. von 0 auf 255.

1.5 Bit-Operationen – Die vier Grundoperationen

Übersicht:

Operation	Operator	Code	Wirkung
Bit setzen	OR	`x |= mask`	0→1, 1→1
Bit löschen	AND NOT	`x &= ~mask`	1→0, 0→0
Bit toggeln	XOR	`x ^= mask`	0→1, 1→0
Bit prüfen	AND	`x & mask`	Ergebnis $\neq 0$ wenn gesetzt

Tabelle 7: Die vier Bit-Grundoperationen

Bit setzen (OR)

Prinzip: `x | 0 = x` (bleibt), `x | 1 = 1` (wird gesetzt)

```
uint8_t ledState = 0b00000000;
ledState |= 0b00010000;  // Bit 4 setzen

// Visualisierung:
//   0b00000000
// | 0b00010000
// ------------
//   0b00010000
```

Anwendung: LED einschalten

```
ledState |= (1u << 4);  // Bit 4 setzen -> LED 5 an (MSB-first)
```

Bit löschen (AND NOT)

Prinzip: `x & 1 = x` (bleibt), `x & 0 = 0` (wird gelöscht)

```
uint8_t ledState = 0b00010000;
ledState &= ~0b00010000;  // Bit 4 loeschen

// Visualisierung:
// ~0b00010000 = 0b11101111
//
//   0b00010000
// & 0b11101111
// -----------
//   0b00000000
```

Anwendung: LED ausschalten

```
ledState &= ~(1u << 4);  // Bit 4 loeschen -> LED 5 aus
```

Bit toggeln (XOR)

Prinzip: `x ^ 0 = x` (bleibt), `x ^ 1 = ~x` (wechselt)

```
uint8_t ledState = 0b00010000;
ledState ^= 0b00010000;  // Bit 4 toggeln -> 0

ledState ^= 0b00010000;  // Bit 4 toggeln -> 1
```

Anwendung: Blinken ohne Zustandsvariable

```
ledState ^= (1u << 4);  // Jeder Aufruf wechselt den Zustand
```

Bit prüfen (AND)

Prinzip: Ergebnis ist 0 wenn Bit nicht gesetzt, sonst $\neq 0$

```
uint8_t buttonState = 0b11011111;  // Bit 5 = 0 (Button gedrueckt)

if (buttonState & 0b00100000) {
    // Bit 5 ist gesetzt -> Button NICHT gedrueckt
} else {
    // Bit 5 ist 0 -> Button gedrueckt (active-low)
}
```

Anwendung: Button-Zustand abfragen

```
bool isPressed = (buttonState & (1u << 5)) == 0;  // Active-low
```

1.6 Warum `1u` statt `1`?

Problem: Der Literal `1` ist vom Typ `int` (signed). Bei Bit-Shifts kann das zu undefiniertem Verhalten führen.

```
// Problematisch:
1 << 31  // Undefiniert! Signed overflow

// Sicher:
1u << 31  // OK, unsigned
```

Regel

Verwende immer `1u` bei Bit-Operationen:

```
ledState |= (1u << bitPos);     // Setzen
ledState &= ~(1u << bitPos);    // Loeschen
if (state & (1u << bitPos))     // Pruefen
```

1.7 Maske erstellen – Vom Bit zur Maske

Formel: `mask = 1u « bitPosition`

Bit-Position	Shift	Maske (binär)	Maske (hex)
0	`1u « 0`	`0b00000001`	`0x01`
1	`1u « 1`	`0b00000010`	`0x02`
2	`1u « 2`	`0b00000100`	`0x04`
3	`1u « 3`	`0b00001000`	`0x08`
4	`1u « 4`	`0b00010000`	`0x10`
5	`1u « 5`	`0b00100000`	`0x20`
6	`1u « 6`	`0b01000000`	`0x40`
7	`1u « 7`	`0b10000000`	`0x80`

Tabelle 8: Masken für Bit-Positionen 0–7

Merkregel

Die Hex-Werte folgen dem Muster 1, 2, 4, 8 (pro Nibble).

1.8 MSB-first Mapping – Von der ID zur Bit-Position

Projektstandard: Das Selection Panel verwendet MSB-first. ID 1 liegt in Bit 7, ID 8 in Bit 0.

Formeln:

```
uint8_t byteIndex   = (id - 1) / 8;
```

```
uint8_t bitPosition = 7 - ((id - 1) % 8);  // MSB-first!
uint8_t mask        = 1u << bitPosition;
```

Herleitung am Beispiel – ID 5:

```
id = 5

byteIndex   = (5 - 1) / 8       = 4 / 8 = 0
bitPosition = 7 - ((5 - 1) % 8) = 7 - 4  = 3
mask        = 1u << 3           = 0x08

// Visualisierung:
// Byte 0:  ID  1   2   3   4   5   6   7   8
//          Bit 7   6   5   4   3   2   1   0
//                              ^
//                           ID 5 -> Bit 3 -> Maske 0x08
```

Mapping-Tabelle (10 IDs):

ID	Byte	Bit	Maske
1	0	7	0x80
2	0	6	0x40
3	0	5	0x20
4	0	4	0x10
5	0	3	0x08
6	0	2	0x04
7	0	1	0x02
8	0	0	0x01
9	1	7	0x80
10	1	6	0x40

Tabelle 9: MSB-first Mapping für 10 IDs

1.9 Praktische Funktionen

LED setzen/löschen/abfragen

```
#define LED_COUNT 10
#define LED_BYTES ((LED_COUNT + 7) / 8)

uint8_t ledState[LED_BYTES] = {0};

void setLed(uint8_t id) {
    if (id == 0 || id > LED_COUNT) return;
    uint8_t byteIdx = (id - 1) / 8;
    uint8_t bitPos  = 7 - ((id - 1) % 8);
    ledState[byteIdx] |= (1u << bitPos);
```

```
}

void clearLed(uint8_t id) {
    if (id == 0 || id > LED_COUNT) return;
    uint8_t byteIdx = (id - 1) / 8;
    uint8_t bitPos  = 7 - ((id - 1) % 8);
    ledState[byteIdx] &= ~(1u << bitPos);
}

bool getLed(uint8_t id) {
    if (id == 0 || id > LED_COUNT) return false;
    uint8_t byteIdx = (id - 1) / 8;
    uint8_t bitPos  = 7 - ((id - 1) % 8);
    return (ledState[byteIdx] & (1u << bitPos)) != 0;
}
```

Button prüfen (Active-Low)

```
bool isButtonPressed(uint8_t id, const uint8_t* raw) {
    if (id == 0 || id > BTN_COUNT) return false;
    uint8_t byteIdx = (id - 1) / 8;
    uint8_t bitPos  = 7 - ((id - 1) % 8);

    // Active-low: gedrueckt wenn Bit = 0
    return (raw[byteIdx] & (1u << bitPos)) == 0;
}
```

One-Hot setzen

```
void setOneHot(uint8_t selectedId, uint8_t* out, size_t bytes) {
    memset(out, 0, bytes);  // Alle Bits loeschen

    if (selectedId >= 1 && selectedId <= LED_COUNT) {
        uint8_t byteIdx = (selectedId - 1) / 8;
        uint8_t bitPos  = 7 - ((selectedId - 1) % 8);
        out[byteIdx] = 1u << bitPos;
    }
}
```

1.10 Last-Byte-Maskierung

Problem: Bei 10 LEDs brauchst du 2 Bytes (16 Bits), aber nur 10 sind gültig. Die Bits 5–0 im zweiten Byte sind „überzählig“.

Berechnung:

```
uint8_t validBits = LED_COUNT % 8;
if (validBits == 0) validBits = 8;
```

```
3
4 // MSB-first: die obersten validBits sind gueltig
5 uint8_t LAST_MASK = 0xFF << (8 - validBits);
```

Count	validBits	LAST_MASK	Gültige Bits
8	8	`0xFF`	alle
10	2	`0xC0`	Bit 7, 6
12	4	`0xF0`	Bit 7, 6, 5, 4
100	4	`0xF0`	Bit 7, 6, 5, 4

Tabelle 10: Last-Byte-Masken für verschiedene Kanalzahlen

Anwendung:

```
1 // LEDs: ungueltige Bits auf 0 setzen (bevor an HC595 gesendet)
2 ledState[LED_BYTES - 1] &= LED_LAST_MASK;
3
4 // Buttons: ungueltige Bits auf 1 setzen (active-low idle)
5 raw[BTN_BYTES - 1] |= ~BTN_LAST_MASK;
```

1.11 Zusammenfassung – Merkregeln

Zahlensysteme

Situation	Verwende
Zählwerte, Indizes	Dezimal: `10`
Einzelne Bits sichtbar	Binär: `0b10000000`
Byte-Werte	Hex: `0x80`

Tabelle 11: Zahlensystem-Wahl

Bit-Operationen

Aufgabe	Code
Bit setzen	`x |= (1u << n)`
Bit löschen	`x &= ~(1u << n)`
Bit toggeln	`x ^= (1u << n)`
Bit prüfen	`if (x & (1u << n))`

Tabelle 12: Bit-Operationen Kurzreferenz

Formel	Berechnung
Byte-Anzahl	`(count + 7) / 8`
Byte-Index	`(id - 1) / 8`
Bit-Position	`7 - ((id - 1) % 8)`
Maske	`1u << bitPosition`

Tabelle 13: MSB-first Mapping-Formeln

Bit	7	6	5	4	3	2	1	0
Hex	0x80	0x40	0x20	0x10	0x08	0x04	0x02	0x01
ID (MSB-first)	1	2	3	4	5	6	7	8

Tabelle 14: Hex-Masken und IDs Schnellreferenz

MSB-first Mapping

Hex-Masken (auswendig)

1.12 Weiterführende Dokumentation

`docs/bit-mapping-schieberegister.md` Detailliertes Mapping

`firmware/docs/overview.md` Projektübersicht

`firmware/include/hw/io_counts.h` SSOT für Counts/Masks

2 Bit-Mapping – Schieberegister verstehen

2.1 Begriffsrahmen und Zielsetzung

Das Selection Panel verwendet Schieberegister, um 10–100 Buttons und LEDs mit wenigen GPIO-Pins anzusteuern. Die zentrale Herausforderung: Wie ordnen wir eine Button-ID (z. B. „Button 7“) einem konkreten Bit in einem Byte-Array zu?

Begriff	Definition
Bit-Mapping	Zuordnungsregel zwischen logischer ID und physischer Bit-Position
MSB-first	Most Significant Bit first – Bit 7 wird zuerst übertragen/gelesen
Active-Low	Logisch aktiv bei Pegel 0 (GND)
Active-High	Logisch aktiv bei Pegel 1 (VCC)
One-Hot	Genau ein Bit gesetzt, alle anderen 0
Kaskadierung	Mehrere ICs in Reihe geschaltet (Daisy-Chain)

Tabelle 15: Begriffsrahmen Bit-Mapping

Ziel: Eindeutige, skalierbare Zuordnung von IDs zu Bits – funktioniert identisch für 10 wie für 100 Kanäle.

2.2 Das Grundproblem – IDs vs. Bytes

Problem: Die Firmware-Logik arbeitet mit Button-IDs (1, 2, 3, ..., 100). Die Hardware arbeitet mit Byte-Arrays. Zwischen beiden muss eine eindeutige Übersetzung existieren.

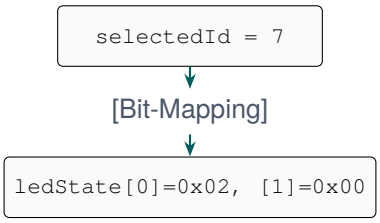


Abbildung 2: Übersetzung zwischen Logik-IDs und Hardware-Bytes

Regel: IDs sind 1-basiert (1..N), Bytes sind 0-basiert (0..bytes-1), Bits sind 0–7.

Byte-Anzahl berechnen:

```
bytes = (count + 7) / 8
```

Kanäle	Rechnung	Bytes
8	$(8+7)/8 = 1$	1
10	$(10+7)/8 = 2$	2
16	$(16+7)/8 = 2$	2
100	$(100+7)/8 = 13$	13

Tabelle 16: Byte-Anzahl für verschiedene Kanalzahlen

Konsequenz: Die Logik kennt nur IDs. Die Treiber kennen nur Bytes. Das Bit-Mapping ist die Brücke dazwischen.

2.3 MSB-first – Warum Bit 7 zuerst?

Problem: Bei SPI-Kommunikation gibt es zwei Konventionen: MSB-first (Bit 7 zuerst) oder LSB-first (Bit 0 zuerst). Die Wahl beeinflusst das gesamte Mapping.

Entscheidung im Projekt: MSB-first für beide Schieberegister-Typen.

Warum MSB-first? Wenn wir die Bytes in der Reihenfolge senden, wie sie im Array stehen, und MSB-first verwenden, ergibt sich eine intuitive Zuordnung:

```
Array:     [Byte 0] [Byte 1] [Byte 2] ...
Bits:      76543210 76543210 76543210 ...
IDs:       12345678 9...16   17...24   ...
```

Die erste ID (1) landet im höchstwertigen Bit (7) des ersten Bytes (0). Das ist leicht nachvollziehbar und debugbar.

Mapping-Formel (für ID 1..N):

```
byte_index = (id - 1) / 8
bit_position = 7 - ((id - 1) % 8)
mask = 1 << bit_position
```

Herleitung am Beispiel:

Für ID = 7:

- `byte_index = (7-1) / 8 = 0` → Byte 0
- `bit_position = 7 - ((7-1) % 8) = 7 - 6 = 1` → Bit 1
- `mask = 1 « 1 = 0x02`

Für ID = 9:

- `byte_index = (9-1) / 8 = 1` → Byte 1
- `bit_position = 7 - ((9-1) % 8) = 7 - 0 = 7` → Bit 7
- `mask = 1 « 7 = 0x80`

2.4 Buttons (CD4021) – Active-Low lesen

Funktionsweise: Der CD4021 ist ein Parallel-zu-Seriell-Wandler. Er liest 8 parallele Eingänge und schiebt sie seriell zum ESP32.

Active-Low bedeutet:

Physischer Zustand	Pegel am Pin	Bit-Wert
Button losgelassen	HIGH (Pullup)	`1`
Button gedrückt	LOW (nach GND)	`0`

Tabelle 17: Active-Low-Logik beim CD4021

Idle-Erwartung

Wenn kein Button gedrückt ist, liest du `0xFF` pro Byte (alle Bits = 1).

Mapping-Tabelle (10 Buttons):

Beispiel – Button 3 gedrückt:

ID	Byte	Bit	Maske	Gedrückt =
1	0	7	`0x80`	Bit 7 = 0
2	0	6	`0x40`	Bit 6 = 0
3	0	5	`0x20`	Bit 5 = 0
4	0	4	`0x10`	Bit 4 = 0
5	0	3	`0x08`	Bit 3 = 0
6	0	2	`0x04`	Bit 2 = 0
7	0	1	`0x02`	Bit 1 = 0
8	0	0	`0x01`	Bit 0 = 0
9	1	7	`0x80`	Bit 7 = 0
10	1	6	`0x40`	Bit 6 = 0

Tabelle 18: Bit-Mapping für 10 Buttons (MSB-first)

```
Byte 0: 0b11011111 = 0xDF  (Bit 5 = 0)
Byte 1: 0b11111111 = 0xFF  (keine Aenderung)
```

Prüfung im Code:

```
bool isPressed(uint8_t id, const uint8_t* raw) {
    uint8_t byteIdx = (id - 1) / 8;
    uint8_t bitPos  = 7 - ((id - 1) % 8);
    uint8_t mask    = 1 << bitPos;

    // Active-low: gedrueckt wenn Bit = 0
    return (raw[byteIdx] & mask) == 0;
}
```

2.5 LEDs (74HC595) – Active-High schreiben

Funktionsweise: Der 74HC595 ist ein Seriell-zu-Parallel-Wandler. Er empfängt Daten seriell vom ESP32 und gibt sie parallel an 8 Ausgänge aus.

Active-High bedeutet:

Gewünschter Zustand	Bit-Wert	Ergebnis
LED aus	`0`	Ausgang LOW
LED an	`1`	Ausgang HIGH

Tabelle 19: Active-High-Logik beim 74HC595

Mapping-Tabelle (identisch zu Buttons):

One-Hot-Ausgabe:

ID	Byte	Bit	Maske	LED an =
1	0	7	0x80	0x80 0x00
2	0	6	0x40	0x40 0x00
...	...	...	...	...
8	0	0	0x01	0x01 0x00
9	1	7	0x80	0x00 0x80
10	1	6	0x40	0x00 0x40

Tabelle 20: Bit-Mapping für 10 LEDs (MSB-first)

```
void setOneHot(uint8_t selectedId, uint8_t* ledState, size_t bytes) {
    // 1. Alle LEDs aus
    memset(ledState, 0x00, bytes);

    // 2. Genau eine LED setzen (wenn ID > 0)
    if (selectedId >= 1 && selectedId <= LED_COUNT) {
        uint8_t byteIdx = (selectedId - 1) / 8;
        uint8_t bitPos  = 7 - ((selectedId - 1) % 8);
        ledState[byteIdx] |= (1 << bitPos);
    }
}
```

Beispiel – LED 9 an:

```
Byte 0: 0b00000000 = 0x00
Byte 1: 0b10000000 = 0x80  (Bit 7 = 1)
```

2.6 Last-Byte-Maskierung – Ungültige Bits behandeln

Problem: Bei 10 Kanälen brauchst du 2 Bytes (16 Bits), aber nur 10 sind gültig. Die Bits 5–0 im zweiten Byte sind „überzählig“. Wenn sie durch Speicherfehler oder Bugs gesetzt werden, könnte das System falsch reagieren.

Regel: Ungültige Bits müssen maskiert werden – bei Buttons auf 1 (idle), bei LEDs auf 0 (aus).

Berechnung der Maske:

```
uint8_t valid_bits = count % 8;
if (valid_bits == 0) valid_bits = 8;  // Volle 8 Bits

// MSB-first: die obersten valid_bits sind gueltig
uint8_t LAST_MASK = 0xFF << (8 - valid_bits);
```

Beispiele:

Anwendung:

Count	valid_bits	LAST_MASK	Binär	Gültige Bits
8	8	`0xFF`	`11111111`	alle
10	2	`0xC0`	`11000000`	Bit 7, 6
12	4	`0xF0`	`11110000`	Bit 7, 6, 5, 4
100	4	`0xF0`	`11110000`	Bit 7, 6, 5, 4

Tabelle 21: Last-Byte-Masken für verschiedene Kanalzahlen

```
// Nach dem Lesen (Buttons): ungueltige Bits auf 1 setzen
raw[last_byte] |= ~BTN_LAST_MASK;

// Vor dem Schreiben (LEDs): ungueltige Bits auf 0 setzen
led[last_byte] &= LED_LAST_MASK;
```

Warum ist das wichtig?

Bei 10 LEDs und `LAST_MASK = 0xC0`: Wenn `led[1] = 0xFF` (durch Bug), würden nach Maskierung nur `led[1] = 0xC0` an den HC595 gehen. Ohne Maskierung: 6 „Phantom-LEDs“ würden leuchten.

2.7 Kaskadierung – Mehrere ICs verketten

Problem: Ein CD4021 oder 74HC595 hat nur 8 Kanäle. Für 10, 50 oder 100 Kanäle müssen mehrere ICs in Reihe geschaltet werden.

CD4021-Kette (Input)

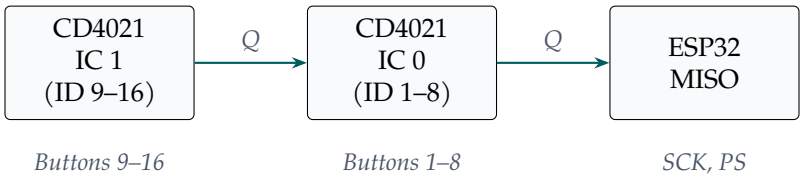


Abbildung 3: CD4021-Kaskadierung für Button-Input

Reihenfolge: Das erste gelesene Bit kommt vom IC, der direkt am ESP32 hängt (IC 0). Die Daten der kaskadierten ICs werden „durchgeschoben“.

74HC595-Kette (Output)

Reihenfolge: Die zuerst gesendeten Bits landen im hintersten IC der Kette. Byte 0 geht an IC 0, Byte 1 an IC 1 usw.

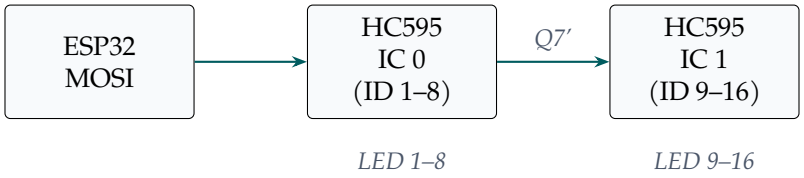


Abbildung 4: 74HC595-Kaskadierung für LED-Output

Typischer Fehler

Wenn die ICs falsch herum verkabelt sind, erscheinen die IDs „gespiegelt“ – Button 1 löst LED 8 aus, Button 9 löst LED 16 aus.

2.8 Vollständiges Beispiel – Button 7 drücken

Wir verfolgen den Datenfluss, wenn Button 7 gedrückt wird:

1. Hardware-Zustand:

```
Button 7 zieht Pin auf GND -> Bit 1 von Byte 0 wird 0
```

2. CD4021 liest (2 Bytes):

```
raw[0] = 0b11111101 = 0xFD  (Bit 1 = 0)
raw[1] = 0b11111111 = 0xFF
```

3. Logik erkennt Button 7:

```
id = 7
byteIdx = (7-1)/8 = 0
bitPos  = 7 - ((7-1) % 8) = 7 - 6 = 1
mask    = 0x02

// Pruefung: (raw[0] & 0x02) == 0?
// -> (0xFD & 0x02) = 0x00 -> JA, gedrueckt!
```

4. Selection-Logik setzt `selectedId = 7`

5. One-Hot erzeugt LED-Bytes:

```
led[0] = 0x00;  // clear
led[1] = 0x00;

// Set Bit fuer ID 7:
byteIdx = 0
bitPos  = 1
led[0] |= (1 << 1) = 0x02

// Ergebnis:
```

```
led[0] = 0b00000010 = 0x02
led[1] = 0b00000000 = 0x00
```

6. HC595 schreibt:

```
IC 0 erhaelt 0x02 -> Ausgang Q1 = HIGH -> LED 7 leuchtet
IC 1 erhaelt 0x00 -> alle Ausgaenge LOW
```

2.9 Skalierung – 10 → 100 ohne Code-Änderung

Änderungen in `io_counts.h`:

```
// Vorher (10 Kanaele)
#define BTN_COUNT  10
#define LED_COUNT  10

// Nachher (100 Kanaele)
#define BTN_COUNT  100
#define LED_COUNT  100
```

Automatisch abgeleitete Werte:

Parameter	10 Kanäle	100 Kanäle
BTN_BYTES	2	13
LED_BYTES	2	13
BTN_LAST_MASK	0xC0	0xF0
LED_LAST_MASK	0xC0	0xF0

Tabelle 22: Automatisch abgeleitete Parameter bei Skalierung

Keine Änderungen nötig in:

- Bit-Mapping-Formeln (arbeiten mit beliebigen IDs)
- Treiber-Code (iteriert über `*_BYTES`)
- Logik-Code (arbeitet mit IDs, nicht Bytes)

Konsequenz

Die Architektur ist skalierbar, solange die SSOT-Werte angepasst werden.

2.10 Fehlerbehebung

Diagnose-Befehl:

Symptom	Wahrscheinliche Ursache	Diagnose
RAW = 0xFF (immer)	Pullups fehlen oder PS/LOAD-Signal falsch	Oszilloskop an PS/LOAD
RAW = 0x00 (immer)	Kurzschluss nach GND	Durchgangsprüfung
Falsche ID erkannt	MSB/LSB vertauscht oder Kaskadierung falsch	E2E-Test mit einzelnen Buttons
LEDs spiegelverkehrt	Byte-Reihenfolge beim HC595 falsch	Verkabelung Q7′ prüfen
Letzte IDs reagieren nicht	LAST_MASK fehlt oder falsch	Masken-Berechnung prüfen

Tabelle 23: Fehlerbehebung Schieberegister

```
pio test -d firmware -e seeed_xiao_esp32s3_e2e -f test_e2e_hw -v
```

2.11 Weiterführende Dokumentation

`firmware/include/hw/io_counts.h` SSOT für Counts/Masks

`firmware/docs/wiring.md` Hardware-Verdrahtung

`firmware/docs/architecture.md` Schichtenmodell

`firmware/docs/debug-playbook.md` Erweiterte Fehlerbehebung